DM574 - Final Project Part 2 in LaTeX

Mathias Byrk Størner[1], Mathias Westergaard Nielsen[1], Phillip Helmer Dellgren[1], and Simon Løkkegaard[1]

[1]Computer Science (Datalogi), University of Southern Denmark

December 19th, 2025

1 Introduction

In this report, each function written will be explained in detail, such that the reader is made aware of the thoughts associated with them, and why we ultimately ended up choosing that specific solution to each of our functions. The following report contains two subsections, one briefly explaining our implementation of helper functions, and then a final subsection explaining each function that is requested for completion of the project itself.

2 Files

Relevant files for this project are located at:

Enigma Part 2

- /Enigma2.py/
  - Dependencies:
    - /EnigmaModel.py/
    - /EnigmaConstants.py/

Project Report

- /report_files/report.pdf/
  - Our group report in PDF format.
- /report_files/report.tex/
  - Our group report sourcecode in TEX format.

Git Log

- /git.Log
  - Log of all git commits

3 Helper functions

3.1 Imports

We started by importing everything from the "EnigmaModel.py"and "Enigma-Constants.py"files, in order to reduce the overall amount of duplicate code.

3.2 carry_over

A new carry over function was written, as it is more accessible when only rotors are being used, and no other methods of the class were to be used. In short, this solution is much more simple. This is also an important helper function, as it is used for brute forcing and rotor rotations. This logic is much more suited to the two functions "find_rotors"and "decrypt".

A new variable is created, which is used to store each rotor state in a list. To access the rotors, we used indices to grab the value of the desired rotor. If the fast rotor has reached the end of the alphabet, it will reset, followed by using the same logic on the next rotor, until we reach the slow rotor. If it turns out that the desired rotor hasn't reached the end, it will just increase by one - much like the original logic of the *advance* method.

3.3 convert_rotor_to_str

This helper function takes a list of integers, and transform it into a string. The reason we built this function, was that the *encrypt* function needs a rotor setting as a string, e.g. "AAA", but if you wish to advance a rotor, it is easier when the rotor is in the form of a list of integers, as there has to be no further conversion or indexing.

3.4 convert_rotor_to_int

This function translates the rotor settings given as a string, into a list of integers. This is required for the *encrypt* function, as we wanted to reuse some of the logic from the *EnigmaModel*.

4 Project Functions

4.1 encrypt

Our thought process behind the creation of the *encrypt* function was, that since we already understood the logic and behaviour of the Enigma machine from part 1, we decided to reuse some of the code and extend it with the necessary additions. The primary thing we had to make sure of, was that the function takes any kind of message, and a rotor setting, and then returns an encrypted message.

The function initializes three independent rotors, which are each assigned a specific offset based on the function input, e.g. "ABC". The string is converted into a list of integers using the helper function *"convert_rotor_to_int"*. We chose this approach as we could reuse as much of the existing logic and code from *EnigmaModel* and still ensure that the function behaved like the Enigma machine.

Subsequently, we had to ensure that our function could take a message string containing spaces and lowercase letters, and still return a valid encrypted message, both for ease of use, and to prevent mistakes, although the input is assumed to not contain any of these. This part was quickly solved by using Pythons *join* and *upper* string methods.

Afterwards, we reused core parts of our previous code from the *EnigmaModel*, which includes the rotors' carry over and the signal path through the rotors.

We tested this function, by running the code from part 1, with a chosen rotor setting and typing in the exact same sentence, and writing down the encrypted message letter for letter. With the encrypted cipher from the part 1 Enigma Machine, we encrypted the same message using the same rotor settings, and compared the two. We found that they were equal, during multiple tests.

4.2 find_rotors

For this function, we agreed that since we have the original message and what it will be after encryption, we'd create a while loop that encrypts the message with rotor settings advancing every time, until the cipher and the encrypted message were equal to one another.

We found that using a while loop to loop through all possible rotor settings was the most optimal, as we did not know exactly how many iterations were needed.

To briefly summarize, our solution was to loop through the rotorsettings from "AAA"to "ZZZ", encrypt the message with the given setting, then check if the encrypted message is equal to the cipher. Once our equality operator returns True, we return the rotor settings.

To test this function, we used the *encrypt* function to create some test cases, in which we saved the original message, the encrypted message, and the rotor settings. We used the test cases by passing a message and a cipher text to the *find_function*, and once it had found the correct rotor settings in all of the cases, we knew it was functioning as intended.

4.3 decrypt

The decrypt function is comprised of two main parts: the word list, and a brute-force method to finding the most accurate guess that we were able to produce.

During the initial idea-phase of this function, we discussed various ways to match words to the cipher, including how we would seperate words, and get the most accurate guess possible. Due to the contraints of the task, such as the cypher being encrypted using exclusively capital latin letters, we found no way to seperate each word with 100% accuracy. Therefore, we decided to simply guess each word in whichever order they were given in our word list. We concluded that the best way to split words, due to our choice of using accuracy estimation for the best guess, was to guess the longest words first, and then move on to shorter words after.

Writing the function, we decided to take each word from the "words10k.txt"file and append to a list. Furthermore we enforced everthing appended to the list to uppercase and remove single letters. We experienced that we would get 100% accuracy, due to the alphabet also being contained within the words10k.txt file. Another issue we faced, was how we were going to handle that some words could wrongfully be split into other words, like "there"→ "the"+ "re", meaning two characters would not be included in our accuracy estimation. This lead to us deciding that we sorted the list in a descending order, ordered by length of the word.

After this, the *cipher* is passed through two methods to ensure that there are no spaces, and all letters are capital, although we assume there are only latin letters and no special characters, other than spaces, in the message.

We then create a while loop to go through all rotor settings, and initiliaze some variables used for conversion and storing the decrypted message.

After this, we go through each element in our word list, and if the chosen word of the word list is in the cipher, it will count the amount of characters, and remove the word from the cipher. Once all words of the word list have been checked, if there are at least one word which was matched with the cipher, the percentage accuracy is determined by taking the matching amount of characters divided by the initial length of the cipher, and getting a percentage accuracy guess. If this guess is better than the previous best guess, it will be replaced, and move on to the next rotor using the carry_over helper function.

Finally, it will return a string containing formatting for printing the rotor, mes-

sage and the accuracy percentage.

Wwe tested this function by creating some test cases using the encrypt function. We created different test cases by varying sentence length, and which words were used, specifically some that may not be in our word list. For each of the test cases, we ran them through the function, and printed the output. During this test phase, we encountered multiple answers with 100% accuracy. We were aware of single-character words which could have posed an issue, however we discovered that the word list contained the entire alphabet as single-character words, meaning every message would end up with 100% accuracy. This led to the decision of removing all single-character words. We also discovered long runtimes for the program, which we reduced by using a smaller words list of only 10.000 of the most common words, rather than the previous $\tilde{1}$13.000 words. It was also found that the removal of words from the cipher did not function as intended, as it allowed words to be formed by letters that were not previously connected. To mitigate this error, a space is inserted instead of the word, such that no new words could be formed by the deletion of a found word.